Praktikum 2. Matematika ML pada Ribuan Observasi Sensor

Dataset real: AirQualityUCI • Fokus: representasi data → prediksi → error → loss → objective → optimisasi

A. Capaian dan Bukti Belajar

- Menjelaskan satu observasi sebagai vektor, banyak observasi sebagai matriks X, dan target sebagai y.
- Menunjukkan peran weight dan intercept pada prediksi linear sederhana.
- Mengukur jarak sebelum dan sesudah scaling serta menjelaskan mengapa skala mengubah geometri algoritma.
- Membaca mean, median, standard deviation, distribusi, dan probabilitas empiris pada data nyata.
- Membedakan error, loss, dan objective dengan bukti numerik.
- Menjalankan gradient descent, menelusuri satu iterasi penuh, dan menjelaskan pengaruh learning rate.
- Membedakan parameter yang dipelajari model dari hyperparameter yang mengatur proses training.

B. Peta Praktikum: Mengapa Urutannya Seperti Ini?

Langkah	Pertanyaan yang dijawab
Data → X dan y	Informasi apa yang masuk ke model dan apa yang ingin diprediksi?
Weight + intercept	Bagaimana feature digabung menjadi prediction?
Distance + scaling	Apa arti “mirip”, dan apakah skala angka membuat satu feature mendominasi?
Statistik + distribusi	Seperti apa pusat, sebaran, bentuk, dan variasi data?
Error	Seberapa jauh satu prediction dari target?
Loss	Bagaimana satu kesalahan diberi skor?
Objective	Bagaimana banyak loss digabung menjadi satu angka yang dapat diminimalkan?
Gradient	Ke arah mana parameter harus diubah?
Learning rate	Seberapa besar langkah perubahan parameter?

C. Menyiapkan Data tanpa Menyembunyikan Masalahnya

Tujuan. Membentuk working view yang numerik dan dapat dianalisis, sambil tetap mempertahankan jejak bahwa nilai -200 pada dataset adalah sentinel missing.

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

pd.set_option("display.max_columns", 50)
np.set_printoptions(precision=4, suppress=True)

print("Library siap.")

URL = (
    "https://raw.githubusercontent.com/PacktPublishing/"
    "Time-Series-Analysis-with-Python-Cookbook/refs/heads/main/"
    "datasets/Ch2/AirQualityUCI.csv"
)

candidates = [
```

```
    Path("data/AirQualityUCI.csv"),
    Path("../data/AirQualityUCI.csv"),
    Path("AirQualityUCI.csv")
]
local_file = next((p for p in local_candidates if p.exists()), None)
source = str(local_file) if local_file is not None else URL

print("Sumber data:", source)

df_raw = pd.read_csv(source, sep=";")
df_raw = df_raw.dropna(axis=1, how="all")

print("Shape data mentah:", df_raw.shape)
print("\nKolom:")
print(df_raw.columns.tolist())

display(df_raw.head())
```

Pertanyaan 1 :

1. Satu baris pada dataset ini mewakili apa?
2. Apa perbedaan CO(GT) dan PT08.S1(CO) dari sudut pandang peran data?
3. Mengapa Date dan Time penting walaupun tidak menjadi target pada eksperimen ini?

```
df = df_raw.copy()

time_clean = (
    df["Time"]
    .astype(str)
    .str.strip()
    .str.replace(".", ":", regex=False)
)

df["datetime"] = pd.to_datetime(
    df["Date"].astype(str).str.strip() + " " + time_clean,
    dayfirst=True,
    errors="coerce"
)

numeric_cols = [c for c in df.columns if c not in ["Date", "Time", "datetime"]]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors="coerce")

print(df.dtypes)
print("\nDatetime gagal diparsing:", df["datetime"].isna().sum())
```

Pada dataset ini, -200 digunakan sebagai kode untuk **missing value**. Secara sintaks ia terlihat sebagai angka, tetapi secara **semantik** berarti *nilai tidak tersedia*. Kita mempertahankan df sebagai *raw view*, lalu membuat analysis sebagai *analysis view* dengan -200 → NaN.

```
sentinel_count = (df[numeric_cols] == -200).sum()
sentinel_pct = sentinel_count / len(df) * 100

sentinel_summary = pd.DataFrame({
    "jumlah_-200": sentinel_count,
    "persen_-200": sentinel_pct
}).sort_values("jumlah_-200", ascending=False)

display(sentinel_summary)

# Analysis view
analysis = df.copy()
analysis[numeric_cols] = analysis[numeric_cols].replace(-200, np.nan)
```

```
print("Missing setelah -200 -> NaN:")
display(
    analysis[numeric_cols]
    .isna()
    .sum()
    .sort_values(ascending=False)
    .to_frame("missing")
    .head(15)
)
```

D. Dari Satu Instance ke Vektor, Matriks X, dan Target y

Untuk eksperimen matematika, kita memakai:

- target y = CO(GT);
- kandidat feature: PT08.S1(CO), T, RH, AH.

Kita tidak menyatakan bahwa feature tersebut adalah pilihan model final. Fokusnya adalah memahami representasi matematis.

```
work = analysis[
    ["datetime", "CO(GT)", "PT08.S1(CO)", "T", "RH", "AH"]
].copy()

math_df = (
    work
    .dropna(subset=["CO(GT)", "PT08.S1(CO)"])
    .sort_values("datetime")
    .reset_index(drop=True)
)

feature_cols = ["PT08.S1(CO)", "T", "RH", "AH"]

X = math_df[feature_cols].to_numpy(dtype=float)
y = math_df["CO(GT)"].to_numpy(dtype=float)

print("X.shape =", X.shape)
print("y.shape =", y.shape)
print("\nVektor instance pertama:")
print(X[0])
print("\nTarget instance pertama:")
print(y[0])
```

Pertanyaan 2 :

1. X.shape = (____, ____)
2. Artinya terdapat _____ observasi/instance dan _____ feature.
3. Satu baris X adalah _____.
4. Satu kolom X adalah _____.
5. y berisi _____.

E. Weight, Dot Product, dan Intercept: Dari Feature menjadi Prediksi

Model linear sederhana dapat ditulis sebagai: **y_hat = b + x @ w**. Bagian x @ w adalah penjumlahan kontribusi berbobot setiap feature /Dot Product. Angka b adalah intercept: nilai dasar yang menggeser seluruh prediksi sebelum kontribusi feature ditambahkan.

Makna intercept yang perlu diingat

Pada feature mentah, b secara matematis adalah prediksi ketika semua feature bernilai 0, tetapi kondisi itu belum tentu masuk akal secara fisik. Setelah feature distandardisasi, x_scaled = 0 berarti feature berada pada rata-rata training; dalam kondisi ini intercept jauh lebih mudah ditafsirkan sebagai level prediksi dasar pada kondisi rata-rata.

Demo kontribusi (parameter sengaja dipilih untuk ilustrasi, belum hasil training).

```
# Demonstrasi dot product pada satu instance
# Kita menggunakan feature yang sudah distandardisasi hanya untuk demonstrasi
matematika.
dot_df = math_df.dropna(subset=feature_cols).copy()

cut_dot = int(0.70 * len(dot_df))
scaler_demo = StandardScaler().fit(dot_df[feature_cols].iloc[:cut_dot])
X_demo = scaler_demo.transform(dot_df[feature_cols])

x_demo = X_demo[100]
w_demo = np.array([0.50, 0.20, -0.10, 0.30])
b_demo = 2.00

contribution = x_demo * w_demo
y_hat_demo = b_demo + x_demo @ w_demo

print("x =", x_demo)
print("w =", w_demo)
print("Kontribusi x*w =", contribution)
print("Jumlah kontribusi =", contribution.sum())
print("Intercept b =", b_demo)
print("Prediksi =", y_hat_demo)
print("Target instance :",y[100])
```

Pertanyaan 3 :

1. Apa yang dilakukan dot product?
2. Mana yang merupakan nilai dasar?
3. Mana yang merupakan kontribusi feature?
4. Apakah w_demo sudah dipelajari dari data? Jelaskan.

Pada cell di atas, weight sengaja dipilih untuk demonstrasi. Ia bukan hasil training.

Agar makna intercept jelas, kita gunakan satu feature PT08.S1(CO) yang distandardisasi.

Jika:

[$\hat{y}$ = b + w.x_s]

maka saat x_s = 0 (nilai sensor sama dengan mean training), prediksi menjadi:

[$\hat{y}$ = b]

Jadi setelah standardization, intercept memiliki interpretasi yang cukup intuitif: prediksi dasar ketika feature berada tepat di rata-rata training.

```
#  Menyiapkan satu-feature model
one = math_df[["datetime", "CO(GT)", "PT08.S1(CO)"]].dropna().copy()

x_raw = one["PT08.S1(CO)"].to_numpy(dtype=float)
y_one = one["CO(GT)"].to_numpy(dtype=float)

cut = int(0.70 * len(one))
x_train_raw = x_raw[:cut]
y_train = y_one[:cut]

mu_x = x_train_raw.mean()
sd_x = x_train_raw.std(ddof=0)

x_std = (x_raw - mu_x) / sd_x
x_train = x_std[:cut]

print("Mean training PT08.S1(CO):", mu_x)
print("Std training PT08.S1(CO) :", sd_x)
print("Mean x_train setelah standardisasi:", x_train.mean())
```

```
#Intercept menggeser prediksi
idx = min(100, len(x_train)-1)
```

```
x_i = x_train[idx]
y_i = y_train[idx]

w_fixed = 0.8
b_center = y_train.mean()
b_candidates = [b_center - 1.0, b_center, b_center + 1.0]

rows = []
for b in b_candidates:
    pred = b + w_fixed * x_i
    rows.append({
        "b_intercept": b,
        "w": w_fixed,
        "x_std": x_i,
        "prediction": pred,
        "actual_y": y_i,
        "error_y_minus_yhat": y_i - pred
    })

display(pd.DataFrame(rows))
```

Pertanyaan 4 :

1. Apa yang tetap pada tiga baris eksperimen di atas?
2. Apa yang berubah?
3. Jika b naik 1 unit, berapa perubahan prediction?
4. Mengapa kita mengatakan intercept menggeser seluruh prediksi?

F. Distance dan Scaling: Mengapa “Dekat” Tidak Netral?

Kita akan membandingkan Euclidean distance sebelum dan sesudah standardization.

```
# Data lengkap untuk distance
dist_df = math_df.dropna(subset=feature_cols).reset_index(drop=True)
X_dist = dist_df[feature_cols].to_numpy(dtype=float)

i, j = 10, min(500, len(X_dist)-1)

raw_diff = X_dist[i] - X_dist[j]
raw_distance = np.linalg.norm(raw_diff)

profile_scale = pd.DataFrame({
    "min": dist_df[feature_cols].min(),
    "max": dist_df[feature_cols].max(),
    "range": dist_df[feature_cols].max() - dist_df[feature_cols].min(),
    "std": dist_df[feature_cols].std()
})

display(profile_scale)
print("Selisih mentah dua instance:", raw_diff)
print("Euclidean distance mentah:", raw_distance)

# StandardScaler fit hanya pada 70% awal
cut_dist = int(0.70 * len(dist_df))

scaler = StandardScaler()
scaler.fit(dist_df[feature_cols].iloc[:cut_dist])

X_scaled = scaler.transform(dist_df[feature_cols])

scaled_diff = X_scaled[i] - X_scaled[j]
scaled_distance = np.linalg.norm(scaled_diff)
```

```
print("Selisih setelah scaling:", scaled_diff)
print("Euclidean distance setelah scaling:", scaled_distance)
```

Pertanyaan 5 :

1. Feature mana yang paling besar skalanya sebelum scaling?
2. Apakah feature dengan angka terbesar otomatis paling penting?
3. Apa yang berubah setelah scaling?
4. Mengapa scaler sebaiknya di-fit pada training, bukan seluruh data?

G. Statistik, Distribusi

```
#Statistik target CO(GT)
s = math_df["CO(GT)"].dropna()

summary = pd.Series({
    "count": s.count(),
    "min": s.min(),
    "max": s.max(),
    "mean": s.mean(),
    "median": s.median(),
    "std": s.std()
})

display(summary.to_frame("CO(GT)"))

plt.figure(figsize=(9, 4))
plt.hist(s, bins=50, edgecolor="black")
plt.axvline(s.mean(), linestyle="--", label=f"mean={s.mean():.2f}")
plt.axvline(s.median(), linestyle=":", label=f"median={s.median():.2f}")
plt.xlabel("CO(GT)")
plt.ylabel("Frekuensi")
plt.title("Distribusi CO(GT)")
plt.legend()
plt.show()
```

Pertanyaan 6:

1. Apakah mean dan median dekat?
2. Apa yang dapat Anda simpulkan dari bentuk histogram?
3. Mengapa satu nilai mean tidak cukup untuk memahami dataset?

```
## Eksperimen nilai ekstrem
Bandingkan mean dan median sebelum dan sesudah satu nilai dibuat sangat ekstrem.

# Cell 13 — Mean vs median ketika ada nilai ekstrem
sample5 = s.iloc[:5].to_numpy(dtype=float)

extreme5 = sample5.copy()
extreme5[-1] = extreme5[-1] * 10

comparison = pd.DataFrame({
    "kondisi": ["asli", "satu nilai ekstrem"],
    "mean": [sample5.mean(), extreme5.mean()],
    "median": [np.median(sample5), np.median(extreme5)]
})

print("Data asli :", sample5)
print("Data ekstrem:", extreme5)
display(comparison)
```

Pertanyaan 7:
Ringkasan mana yang berubah lebih tajam: mean atau median? Mengapa?

H. Error → Loss → Objective

Definisi operasional

Error/residual menunjukkan selisih prediction dan target. Loss mengubah kesalahan satu instance menjadi skor. Objective menggabungkan loss banyak instance menjadi satu nilai yang menjadi sasaran training. Pada modul ini residual untuk pelaporan didefinisikan r = y - y_hat.

```
#Contoh error dan loss untuk 5 observasi
b0 = y_train.mean()
w0 = 0.0

y_hat0 = b0 + w0 * x_train
error0 = y_train - y_hat0

example = pd.DataFrame({
    "y": y_train[:5],
    "y_hat": y_hat0[:5],
    "error_y_minus_yhat": error0[:5],
    "absolute_error": np.abs(error0[:5]),
    "squared_error": error0[:5]**2
})

display(example)

print("MAE 5 observasi =", example["absolute_error"].mean())
print("MSE 5 observasi =", example["squared_error"].mean())
```

I. Objective Function

```
# Fungsi objective
b_fixed = y_train.mean()

def mse_objective_w(w, x=x_train, y=y_train, b=b_fixed):
    pred = b + w * x
    return np.mean((y - pred)**2)

# Nilai w optimum analitik untuk membantu menentukan rentang visualisasi
w_star = np.mean(x_train * (y_train - b_fixed)) / np.mean(x_train**2)

w_candidates = [0.0, 0.5*w_star, w_star, 1.5*w_star]
table_obj = pd   "w": w_candidates,
    "J(w)_MSE": [mse_objective_w(w) for w in w_candidates]
})

display(table_obj)
print("Perkiraan w dengan objective minimum:", w_star)
```

```
# Visualisasi objective landscape
w_grid = np.linspace(w_star - 3.0, w_star + 3.0, 250)
J_grid = np.array([mse_objective_w(w) for w in w_grid])

plt.figure(figsize=(9, 4))
plt.plot(w_grid, J_grid)
plt.axvline(w_star, linestyle="--", label=f"minimum sekitar w={w_star:.3f}")
plt.xlabel("w")
plt.ylabel("J(w) = MSE training")
```

```
plt.title("Objective function: setiap w mendapat satu skor MSE")
plt.legend()
plt.show()
```

Pertanyaan 8:
1. Apa input fungsi `J(w)`?
2. Apa outputnya?
3. Apa arti satu titik pada kurva?
4. Mengapa training mencari bagian paling rendah?
5. Apa perbedaan **objective function** dengan **gradient descent**?

J. Satu Iterasi Gradient Descent: Intercept dan Weight Sama-sama Diperbaiki

.

```
# Menelusuri SATU iterasi training
b = 0.0
w = 0.0
eta = 0.05

pred = b + w * x_train
err = pred - y_train
mse_before = np.mean(err**2)

db = 2 * np.mean(err)
dw = 2 * np.mean(err * x_train)

b_new = b - eta * db
w_new = w - eta * dw

pred_new = b_new + w_new * x_train
mse_after = np.mean((pred_new - y_train)**2)

trace_one_step = pd.DataFrame({
    "komponen": [
        "b lama", "w lama", "MSE sebelum",
        "gradient db", "gradient dw",
        "learning rate eta", "b baru", "w baru", "MSE sesudah"
    ],
    "nilai": [
        b, w, mse_before,
        db, dw,
        eta, b_new, w_new, mse_after
    ]
})

display(trace_one_step)
```

Pertanyaan 9
1. Apakah MSE turun setelah satu update?
2. Mengapa b dan w sama-sama berubah?
3. Apa fungsi tanda negatif pada rumus update?
4. Apa peran eta?

K. Training Multi-Iterasi dan Learning Rate

```
# Fungsi gradient descent
def run_gradient_descent(x, y, eta=0.05, epochs=80, b_init=0.0, w_init=0.0):
    b = float(b_init)
```

```
    w = float(w_init)
    history = []

    for epoch in range(epochs):
        pred = b + w * x
        err = pred - y
        mse = np.mean(err**2)

        db = 2 * np.mean(err)
        dw = 2 * np.mean(err * x)

        history.append({
            "epoch": epoch,
            "b": b,
            "w": w,
            "mse": mse,
            "db": db,
            "dw": dw
        })

        b -= eta * db
        w -= eta * dw

        if not np.isfinite(mse) or mse > 1e12:
            break

    return pd.DataFrame(history), b, w

# Training satu learning rate
history, b_gd, w_gd = run_gradient_descent(
    x_train, y_train,
    eta=0.05,
    epochs=80
)

display(history.head(10))
print("\nParameter akhir:")
print("b =", b_gd)
print("w =", w_gd)

plt.figure(figsize=(9, 4))
plt.plot(history["epoch"], history["mse"])
plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title("Loss history — gradient descent")
plt.show()
```

```
# Membandingkan learning rate
etas = [0.001, 0.05, 0.5, 1.2]
histories = {}

plt.figure(figsize=(10, 5))

for eta_value in etas:
    h, b_end, w_end = run_gradient_descent(
        x_train, y_train,
        eta=eta_value,
        epochs=80
    )
    histories[eta_value] = h

    safe_mse = np.clip(h["mse"].to_numpy(), 1e-12, None)
    plt.plot(h["epoch"], safe_mse, marker=".", label=f"eta={eta_value}")

plt.yscale("log")
```

```
plt.xlabel("Epoch")
plt.ylabel("MSE (log scale)")
plt.title("Pengaruh learning rate terhadap training")
plt.legend()
plt.show()
```

```
# Ringkasan hasil learning rate
summary_lr = []

for eta_value, h in histories.items():
    summary_lr.append({
        "eta": eta_value,
        "epoch_tercatat": len(h),
        "MSE_awal": h["mse"].iloc[0],
        "MSE_akhir": h["mse"].iloc[-1],
        "MSE_min": h["mse"].min()
    })

display(pd.DataFrame(summary_lr))
```

Pertanyaan 10
Untuk setiap eta, beri label berdasarkan bukti grafik:

- lambat,
- konvergen/stabil,
- agresif,
- berosilasi/divergen.

Jelaskan

Referensi Praktikum

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. — fondasi model, loss, gradient descent, dan workflow ML.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. An Introduction to Statistical Learning with Applications in Python — representasi, statistical learning, dan evaluasi.
- UCI Air Quality dataset / mirror PacktPublishing Time-Series-Analysis-with-Python-Cookbook — data sensor real yang digunakan pada praktikum.